

Milestone Two Narrative

Enhancement One: Software Design and Engineering

Garrett Sikes

SNHU

CS 499 Computer Science Capstone

Artifact: Travlr Getaways backend API (CS 465, Full Stack Development I)

1. Artifact Description

Travlr Getaways: full-stack travel booking app built across CS 465 (Full Stack Development I).

Backend: Node + Express REST API, MongoDB via Mongoose. Frontend for the admin side: Angular SPA. JWT-based auth, issued after a Passport local strategy checks email/password.

The piece being enhanced here is the API layer:

app.js

app_api/routes/index.js

app_api/controllers/authentication.js

2. Justification for Inclusion

I chose this artifact because it was fresh on my mind from the previous semester, I am familiar with the code and I chose the enhancement because even after submitting my work I knew there was more that could be improved on. I did not think much about it, as it was just coursework from a previous semester, but when I saw the opportunity to do so I was kind of excited. I am specializing in Cybersecurity, so this felt more like something I should be doing anyways.

For the enhancement:

Fixed the auth bypass: authenticateJWT called next() unconditionally, outside the async jwt.verify callback, so an invalid token would still let a request through to protected routes. Only a missing header was ever actually blocked. I moved next() inside the success branch.

Closed an additional gap in the DELETE /trips/:tripcode route authenticateJWT, which had no middleware at all. Anyone could realistically delete a trip with no token.

Locked down CORS: app.use(cors()) accepted requests from any origin, I restricted this to a single configured and trusted origin.

Fixed a bug in register(): The failure branch references an undefined err variable, a duplicate email or database hiccup would have thrown a ReferenceError instead of returning a clean response. I wrapped save() in try/catch, added a specific 409 for duplicate email errors.

Added rate limiting (express-rate-limit) and a minimum password length on register/login to make credential stuffing and brute-forcing much harder.

Added .env.example plus a packaging checklist were added so the real .env doesn't accidentally end up in a submission zip again

3. Course Outcomes

When I originally turned in the Travlr Getaways project, everything worked. Authentication worked, and it worked in testing, so nothing about authenticateJWT stood out to me at the time. What changed wasn't that something broke, it was being given the chance to go back and look critically at a piece of the system I'd otherwise never have revisited. Since authentication is one of the first lines of defense for the whole application, I wanted to trace through it more carefully than I had before, and that's when I found that next() was running regardless of whether the token actually verified.

The missing auth check on the DELETE route wasn't something I went looking for. I found it while re-reading the same code with fresh eyes, which taught me that a route can work perfectly and still be a real weakness. Functioning correctly and being secure are not the same thing, and testing alone won't surface that difference if the test never tries the thing the code doesn't guard against.

This has changed how I think about reviewing my own code going forward. I want to be my own toughest critic rather than accepting "it works" as "it's finished." A project that runs is not the same as a project that's actually secure, and the gap between those two is exactly where vulnerabilities live. I would rather find that gap now, in a course, than later, in a system where the cost of missing it is a real data breach.

4. Reflection on the Enhancement Process

For this enhancement, I aimed to meet the two outcomes I identified in my Module One plan: developing a security mindset that anticipates and closes real vulnerabilities, and using well-founded, industry-standard techniques rather than surface-level fixes. Fixing authenticateJWT meant correcting the actual control-flow bug that let invalid tokens through, rather than adding a second check elsewhere to paper over it, which is the well-founded-technique outcome in practice. Finding and closing the unguarded DELETE route is the security-mindset outcome showing up directly: I wasn't looking for that specific gap, but approaching the code with the assumption that something could still be wrong, rather than trusting it because it had always worked, is what let me catch it.

One update to my original plan: I hadn't expected to find a second, unrelated vulnerability during the enhancement itself. That has changed how I think about the security-mindset outcome. It isn't only about fixing the flaw you already know about, it's about staying skeptical of the rest of the code while you're in there. I'd describe that outcome as something I'm continuing to practice rather than something I've fully finished meeting, since treating security as a finished state is exactly the mindset I'm trying to move away from.